

Scaling ML-KEM to (much) higher security

Swechchha Adhikari¹, Vadim Lyubashevsky², and Peter Schwabe^{3,4}

¹ Brigham Young University, Provo, USA

mail@swechchhaa.com

² IBM Research, Zurich, Switzerland

VAD@zurich.ibm.com

³ Max Planck Institute for Security and Privacy, Bochum, Germany

peter@cryptojedi.org

⁴ Radboud University, Nijmegen, The Netherlands

1 Introduction

After a multi-year, multi-round public competition, NIST published its first post-quantum standards FIPS-203 [19], FIPS-204 [18], and FIPS-205 [20] in 2024, specifying ML-KEM, ML-DSA, and SLH-DSA respectively. Interestingly, four of the schemes in the PQC Round 3 Submissions [17] — Kyber [8] (standardized as ML-KEM), Saber [11], NTRU LPRime [6], and FrodoKEM [7] — are closely related lattice KEMs, spanning a spectrum of algebraic structure, from FrodoKEM’s unstructured LWE, through Kyber’s and Saber’s cyclotomic modules, to the prime-degree number fields of NTRU Prime. This spectrum reflects different design principles, commonly understood as structure vs. conservatism. ML-KEM’s module structure gives small keys and fast NTT arithmetic, but is also seen as a potential attack surface, so conservative designs discard it entirely [7]. FrodoKEM, for example, is based on a more conservative hardness assumption of plain LWE, which is why bodies such as BSI, ENISA, and NLNCSA point to it for long-term confidentiality [21]; FrodoKEM was also recently standardized by ISO [14] for this very reason.

It is useful to separate the two notions of conservatism this framing bundles together as they concern two independent (and today, entirely hypothetical) classes of attack. The first arises from the choice of hardness assumption. FrodoKEM rests on unstructured LWE to protect against a structure break, an attack that would exploit the algebraic structure of ring/module LWE, but not apply to plain LWE. The second involves the security margin a scheme carries above its target level, which protects against advances in the best known attacks today. This could include improved lattice reduction, dual attacks, or sharper cost models that would affect the concrete security of every lattice scheme at its current parameters, including FrodoKEM. Both of these hedges are distinct and not to be substituted; structured conservatism does nothing against the second class of attacks and no security margin protects against the first. On top of that, it’s impossible to predict which of the two threats is more likely.

Faced with this uncertainty, increasing the margin has been a historically standard response, provided that sufficient bandwidth is available to accommodate the increase. For computationally secure schemes, concrete security estimate

is the cost of the best attack known at the time, and such estimates evolve as cryptanalysis matures. For instance, the recommended size of an RSA modulus climbed from 512 to 1024 to 2048 bits and beyond as factoring algorithms and available computation advanced [5]. Lattice estimates are not static either. The cost models used for estimation and the attacks have evolved over more than a decade [10,1], and the cost of the best known attacks continues to drift in both directions as improvements are proposed [13,15], scrutinized [12], and refined [9].

If the best known attacks continue to improve, how much margin is required remains open, and the useful question is not whether today’s KEMs carry enough but how much margin they can carry at all. This question arises because additional margin comes at a cost, requiring larger parameters and hence higher bandwidth and runtime, so no scheme can indefinitely withstand advances in cryptanalysis simply by scaling its parameters. For LWE-family KEMs there is a further constraint, since the parameters that determine security also control the decryption failure probability, so increasing the noise to gain a larger security margin necessarily affects correctness.

Crucially, what structure determines is the cost of buying additional margin. At matched security levels, an unstructured scheme already requires roughly an order of magnitude more bandwidth than a module scheme [7,8]. Since these communication costs are considered practical enough for conservative deployments, a module scheme can spend the same bandwidth budget on larger parameters and obtain a conservative security margin while retaining the efficiency advantages of NTT arithmetic.

Recent work has explored this high-margin regime from the perspective of unstructured LWE. KyFrog [16] increases the LWE dimension and adjusts the noise distribution to achieve substantially higher security estimates for much larger ciphertexts. This motivates the complementary question of how much security margin a module KEM can achieve within the communication budget of a conservative unstructured scheme. We answer this question with KAIBURR, a module KEM that scales ML-KEM’s module rank and introduces a generalized noise distribution to preserve correctness. We show that KAIBURR achieves large security margins within FrodoKEM’s communication cost while running substantially faster.

1.1 Our contributions

1.2 Structure of the paper

1.3 Access to artifacts

All software, including parameter analysis scripts, implementation source code, and other artifacts, is publicly available on GitHub.

2 Preliminaries

3 Warmup: Just changing k

The main mechanism to increase bit security in Kyber is to raise the module rank k [2]. Fixing the ML-KEM base ring, noise distribution, and modulus, and varying only k , we measure the decryption-failure probability using the scripts provided by KYBER team.⁵ Throughout we use the centered binomial noise CBD_2 (the noise of ML-KEM-768 and -1024; recall that ML-KEM-512 uses CBD_3). The ranks $k = 3$ and 4 recover the standardized sets ML-KEM-768, and -1024, while $k \geq 5$ is an extrapolation. Table 1 reports the results. Against the 2^{-128} failure bound expected at this security level, the bound holds at $k = 7$ ($2^{-133.7}$) but fails at $k = 8$ ($2^{-116.8}$), so raising k further cannot buy higher security without sacrificing correctness.

These values are computed without ciphertext compression, and therefore are an optimistic estimate. The rounding applied to $\mathbf{u}$ and v in Kyber introduces additional error, which can only increase the true failure probability. Hence, the actual failure probabilities are expected to be higher than those reported here, and the correctness limit on k may be even more restrictive.

Table 1. Decryption-failure probability and public-key and ciphertext sizes as a function of module rank k , at fixed ML-KEM noise and modulus, without ciphertext compression. Ranks $k = 3, 4$ correspond to ML-KEM-768, and -1024.

k	Failure probability	pk (bytes)	ct (bytes)
3	$2^{-299.6}$	1184	1536
4	$2^{-231.1}$	1568	1920
5	$2^{-186.5}$	1952	2304
6	$2^{-155.9}$	2336	2688
7	$2^{-133.7}$	2720	3072
8	$2^{-116.8}$	3104	3456

4 Revisiting the noise distribution

Even though the resulting public key and ciphertext sizes at $k = 7$ remain relatively small, increasing k further is not possible without affecting correctness. Instead, one could reduce the failure probability at large k by increasing the modulus q . We note that all variants of Kyber fix $q = 3329$, which is explicitly justified in the submission documentation to NIST. The modulus $q = 3329$ is the small prime satisfying the NTT-enabling condition $n \mid (q - 1)$, notably weaker than the $2n \mid (q - 1)$ one might expect, which Kyber deliberately does not

⁵ <https://github.com/pq-crystals/security-estimates>

require [3]. It was reduced from the round-1 value 7681 to recover the bandwidth spent by dropping public-key compression and co-tuned to balance ciphertext size, failure probability, and security. Changing q would mean not being able to reuse any existing optimized routines in software or hardware, which sacrifices an important engineering advantage and results in a different scheme altogether.

This is not to say that k is the only parameter that can be modified. Rather, we focus on changes that preserve the *essence* of Kyber: the ring $\mathbb{Z}_q[X]/(X^{256} + 1)$, the NTT implementation and its constants, and the existing analysis framework. The module rank k qualifies trivially. So does the error distribution. Since the noise only affects the error term and does not interact with the arithmetic structure or the NTT, it can be replaced without altering the core implementation. What it changes are only the MLWE hardness estimate and the decryption-failure probability, both of which can be recomputed through the same scripts, over the same ring.

In this sense, the error distribution is another degree of freedom that moves both security and correctness without changing what the scheme is at its core. This is the direction we pursue and where our main contribution lies.

Specifically, to climb past the warmup wall, we need a noise knob that pushes the failure-driving part of the error back down as k grows, while preserving KYBER’s *essence*. Equally important, it should retain the cheap, branchless, constant-time sampling that motivates KYBER’s use of a centered binomial in the first place. This requirement rules out simply using a discrete Gaussian, as in FrodoKEM.

4.1 A parameterized noise distribution

We introduce a one-parameter family $f(t)$ on $\{-2, -1, 0, 1, 2\}$, the same support as CBD_2 , defined by

$$\Pr[\pm 2] = 2^{-t}, \quad \Pr[\pm 1] = \frac{1}{4}, \quad \Pr[0] = \frac{1}{2} - 2^{-(t-1)}.$$

The error at ± 1 is fixed; the single parameter t acts only on the ± 2 tail, thinning it geometrically. A sample is drawn from uniform bits $a_1, \dots, a_t$ as

$$x = (2a_t - 1) \left(1 - a_1 + 2 \prod_{i=1}^{t-1} a_i \right),$$

which is branchless and therefore samples in constant time; we machine-check both constant-time (CT) and speculative constant-time (SCT) in Jasmin [?]. The standard deviation $\sigma(t) = \sqrt{0.5 + 2^{3-t}}$ decreases from 1 at $t = 4$ toward $\sqrt{0.5}$ as t grows.

Crucially, $f(t)$ generalizes Kyber’s own noise distribution. At $t = 4$, the probabilities become

$$\Pr[\pm 2] = \frac{1}{16}, \quad \Pr[\pm 1] = \frac{1}{4}, \quad \Pr[0] = \frac{3}{8},$$

exactly matching CBD_2 , the noise distribution used by ML-KEM-768 and -1024. As $t \rightarrow \infty$, the ± 2 mass vanishes and $f(t)$ approaches CBD_1 .

4.2 Why thinning the tail is the efficient trade

The key property of $f(t)$ is that increasing t reduces the probability of the ± 2 terms exponentially while leaving the ± 1 mass unchanged. As defined above, we have

$$\Pr_{f(t)}[\pm 2] = 2^{-t}, \quad \Pr_{f(t)}[\pm 1] = \frac{1}{4}.$$

So, t provides a direct knob for thinning the tail of the distribution without changing the bulk ± 1 terms. This gives finer control over the noise than the standard centered binomial choices. In particular, CBD_1 and CBD_2 have standard deviations $\sqrt{0.5}$ and 1, respectively, with no centered binomial parameter providing an intermediate distribution. As shown above, $f(t)$ ranges from CBD_2 toward CBD_1 as t increases, giving us finer control over the noise.

This finer control is useful as the module rank k increases. Increasing k increases the amount of error accumulated during encryption and therefore makes the decryption-failure constraint more restrictive. Rather than moving directly from CBD_2 to CBD_1 , we can increase t as k grows, progressively reducing the ± 2 contribution while keeping the ± 1 contribution fixed. For each t , we therefore choose the largest k that satisfies the 2^{-128} failure bound and fits within the FrodoKEM-640 communication budget.

4.3 Selecting t

Since t is the number of uniform bits per sample, byte-aligned values are natural. $t = 4$ recovers the CBD_2 sampler and serves as our anchor; $t = 8$ admits one-byte-per-coefficient SIMD packing; and $t = 6$ sits between them, spending fewer PRF bytes and offering more security per dimension at the cost of less regular packing. The three parameter sets we carry forward are $f(4)$, $f(6)$, and $f(8)$, at $k = 7, 18$, and 24 respectively, which follow from this choice (Table 2). Further details are found in the Implementation section.

Table 2. KAIBURR parameter sets. Classical/quantum security (Core-SVP) and failure probability computed with the Kyber team’s scripts; other parameters as in ML-KEM, compression removed.

Scheme	k	noise	failure	class.	quant.	pk (B)	ct (B)
kaiburr-1792	7	$f(4)$	$2^{-133.7}$	479	434	2720	3072
kaiburr-4608	18	$f(6)$	$2^{-134.1}$	1281	1162	6944	7296
kaiburr-6144	24	$f(8)$	$2^{-139.9}$	1717	1557	9248	9600

4.4 The ceiling of the family

The family $f(t)$ is not unbounded. The lightest noise it can produce is its limit $f(\infty)$, with $\Pr[\pm 1] = \frac{1}{4}$, $\Pr[0] = \frac{1}{2}$, $\Pr[\pm 2] = 0$, and minimal variance $\sigma^2 = 0.5$.

At $n = 256$ and $q = 3329$, the failure probability crosses 2^{-128} at $k = 30$, so $k = 29$ is the largest rank that $f(t)$ can support (Table 3). Since our parameter sets stop at $k = 24$, this ceiling does not bind here, but it marks the point where further thinning of the ± 2 tail is exhausted. Reaching higher security would require another change, such as modifying q , n , or the noise family itself, moving beyond the KYBER structure we seek to preserve.

Table 3. Failure probability under the limiting distribution $f(\infty)$ (minimum variance $\sigma^2 = 0.5$) as k increases, uncompressed. The 2^{-128} bound is first violated at $k = 30$, so $k = 29$ is the ceiling of the $f(t)$ family at $n = 256$, $q = 3329$.

k	Failure probability	pk (bytes)	ct (bytes)
24	$2^{-158.7}$	9248	9600
26	$2^{-146.2}$	10016	10368
28	$2^{-135.5}$	10784	11136
29	$2^{-130.6}$	11168	11520
30	$2^{-126.1}$	11552	11904

5 Implementation specifics

We provide three independent implementations of KAIBURR, each derived from an existing ML-KEM implementation. This includes a portable and vectorized C reference (Section 5.1), a Jasmin implementation carrying constant-time checks (Section ??), and an ACC implementation for the Pavona hardware root of trust (Section ??). All three apply the *same* three modifications to the ML-KEM base:

1. the module rank k is scaled to $\{7, 18, 24\}$ (parameter sets `kaiburr4`, `kaiburr6`, `kaiburr8`; see Table 2);
2. ciphertext and public-key compression are removed, so all coefficients are serialized at full 12-bit precision; and
3. the centered binomial sampler is replaced by the branchless sampler for the noise family $f(t)$, $t \in \{4, 6, 8\}$.

5.1 Portable and vectorized software (C)

Provenance and layout. The C implementation is a fork of the pq-crystals⁶ ML-KEM reference codebase, retaining its split into a portable `ref/` implementation and an AVX2-vectorized `avx2/` implementation. The three parameter sets are selected at compile time by $k \in \{7, 18, 24\}$.

⁶ <https://github.com/pq-crystals/kyber>

The noise sampler. The centered binomial sampler (`cbd.c`) is removed and replaced by `fn.c`, which exposes a single function `poly_fn` realizing Algorithm ???. Each coefficient consumes t uniformly random bits, read least-significant-bit first, and is computed by the branchless expression $(2a_t - 1)(1 - a_1 + 2 \prod_{i < t} a_i)$; the PRF output length is correspondingly `KYBER_NOISE_BYTES` = $t \cdot n / 8$ bytes, giving 128, 192, and 256 bytes per polynomial for $t \in \{4, 6, 8\}$ respectively.

The portable `ref` variant implements three functions `fn4`, `fn6`, and `fn8`, each selected at compile time via the `KYBER_FN` macro based on the parameter set. Each function directly applies the branchless arithmetic, extracting the required t bits per coefficient and computing the final value via bit operations and masking.

The AVX2 variant provides three hand-vectorized kernels that exploit SIMD parallelism. The $t = 8$ kernel processes 32 coefficients per iteration, with each input byte mapping directly to one coefficient; the formula’s product naturally computes as $\prod_{i=1}^7 a_i$ via 7 consecutive bitwise-AND operations, which can be efficiently checked with a single equality comparison to `0xFF` after masking. The $t = 4$ kernel processes 64 coefficients per iteration by extracting two coefficients (4 bits each) from each input byte, splitting into low and high nibbles and then interleaving with sign extension across four 256-bit lanes. The $t = 6$ kernel processes 32 coefficients per iteration by handling the 6-bit packing via a gather-shuffle pattern adapted from ML-KEM’s `cbd3`: it loads 24 bytes, permutes and shuffles to extract four 6-bit fields per 32-bit lane, applies the $f(6)$ arithmetic via vectorized comparisons and arithmetic operations, and then repacks the coefficients. All three kernels are amenable to SIMD because they perform uniform operations across all lanes; the $t = 8$ byte-alignment is the most natural fit for SIMD, requiring no masking or shifting of the product term.

Consequences of removing compression. The compression routines `poly_compress/poly_decompress` and `polyvec_compress/polyvec_decompress` are removed (not stubbed to the identity), together with the associated size macros. The ciphertext now stores both $\mathbf{u}$ and $\mathbf{v}$ at full 12-bit precision. In the `ref` variant this reuses the existing standard-order packer `poly_tobytes`, which packs two coefficients into three bytes via interleaving. In the AVX2 variant, however, `poly_tobytes` is implemented via `ntttobytes_avx` and packs in NTT (bit-reversed) order, which is incorrect for the normal-order ciphertext produced by the inverse NTT. We therefore add new hand-vectorized `cpoly_tobytes/cpoly_frombytes` functions in `indcpa.c` that properly serialize in standard coefficient order. These functions use conditional modular reduction and a shuffle pattern (similar to the gather operation in $f(6)$ sampling) to interleave 16 coefficients into 24 bytes per 256-bit lane, then store the result using standard-order byte layout.

Consequences of scaling `KYBER_K`. Beyond the mechanical parameterization of all size macros and of the $k \times k$ matrix-generation and noise-sampling loops, one critical arithmetic change is required: `polyvec_basemul_acc_montgomery` now calls `poly_reduce` on every iteration of the accumulation loop rather than once at the end. This is necessary because the larger ranks ($k \in \{7, 18, 24\}$)

versus ML-KEM’s maximum $k = 4$) accumulate products that would otherwise overflow the 16-bit accumulator during the multiplication and addition phases. The reduction brings each partial sum back into the valid range $[0, q)$ after each of the k iterations, preventing overflow while maintaining correctness via the properties of modular arithmetic.

Correctness testing. We include a deterministic decapsulation-failure harness (`tests/test_failrate.c`) that runs keypair/encapsulate/decapsulate over many iterations and counts shared-secret mismatches. The test is fully deterministic: it derives randomness from a fixed seed and iteration counter via SHA-3, so failures are reproducible and attributed to specific iteration indices. The harness runs 10^7 iterations by default (customizable via command-line argument) and supports selective testing to target specific implementations (`ref` or `avx2`) or parameter sets (4, 6, or 8). Both the `ref` and `avx2` implementations are tested for all three parameter sets via the `tests/run_failrate.sh` script, which builds and runs all six variants in parallel. This empirically corroborates the analytic decryption-failure probabilities reported in Tables 1, 3, and 2.

6 Benchmarks

We benchmark on three Intel generations following the methodology of [4]: single-core with Hyper-Threading and Turbo Boost disabled, medians over 10001 runs with randomness from Linux `getrandom`.

Acknowledgements

Large parts of the work were done while the first author was employed at MPI-SP.

References

1. Albrecht, M.R., Player, R., Scott, S.: On the concrete hardness of learning with errors. *J. Math. Cryptol.* **9**(3), 169–203 (2015), <http://www.degruyter.com/view/j/jmc.2015.9.issue-3/jmc-2015-0016/jmc-2015-0016.xml> 2
2. Avanzi, R., Bos, J., Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schanck, J.M., Schwabe, P., Seiler, G., Stehlé, D.: CRYSTALS-Kyber — algorithm specifications and supporting documentation. Submission to the NIST Post-Quantum Cryptography Standardization Project (Nov 2017), <https://pq-crystals.org/kyber/data/kyber-specification.pdf>, round 1 submission, 2017-11-30 3
3. Avanzi, R., Bos, J., Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schanck, J.M., Schwabe, P., Seiler, G., Stehlé, D.: CRYSTALS-Kyber (version 2.0) — algorithm specifications and supporting documentation. Submission to round 2 of the NIST Post-Quantum Cryptography Standardization Project (Mar 2019), <https://pq-crystals.org/kyber/data/kyber-specification-round2.pdf>, round 2 submission, 2019-03-30 4

4. Bacelar Almeida, J., Barbosa, M., Barthe, G., Grégoire, B., Laporte, V., Léchenet, J.C., Oliveira, T., Pacheco, H., Quaresma, M., Schwabe, P., Séré, A., Strub, P.Y.: Formally verifying kyber: Episode iv: Implementation correctness. *IACR Transactions on Cryptographic Hardware and Embedded Systems* **2023**(3), 164193 (Jun 2023). <https://doi.org/10.46586/tches.v2023.i3.164-193>, <https://tches.iacr.org/index.php/TCHES/article/view/10960> 8
5. Barker, E.: Recommendation for key management: Part 1 — general. NIST Special Publication 800-57 Part 1, Revision 5, National Institute of Standards and Technology (May 2020). <https://doi.org/10.6028/NIST.SP.800-57pt1r5> 2
6. Bernstein, D.J., Chuengsatiansup, C., Lange, T., van Vredendaal, C.: NTRU prime: Reducing attack surface at low cost. In: Adams, C., Camenisch, J. (eds.) *Selected Areas in Cryptography - SAC 2017 - 24th International Conference, Ottawa, ON, Canada, August 16-18, 2017, Revised Selected Papers. Lecture Notes in Computer Science*, vol. 10719, pp. 235–260. Springer (2017). https://doi.org/10.1007/978-3-319-72565-9_12 1
7. Bos, J.W., Costello, C., Ducas, L., Mironov, I., Naehrig, M., Nikolaenko, V., Raghunathan, A., Stebila, D.: Frodo: Take off the ring! practical, quantum-secure key exchange from LWE. In: *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. pp. 1006–1018. ACM (2016). <https://doi.org/10.1145/2976749.2978425> 1, 2
8. Bos, J.W., Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schanck, J.M., Schwabe, P., Seiler, G., Stehlé, D.: CRYSTALS - kyber: A CCA-secure module-lattice-based KEM. In: *2018 IEEE European Symposium on Security and Privacy, EuroS&P 2018, London, United Kingdom, April 24-26, 2018*. pp. 353–367. IEEE (2018). <https://doi.org/10.1109/EuroSP.2018.00032> 1, 2
9. Carrier, K., Meyer-Hilfiger, C., Shen, Y., Tillich, J.: Assessing the impact of a variant of matzov’s dual attack on kyber. In: Kalai, Y.T., Kamara, S.F. (eds.) *Advances in Cryptology - CRYPTO 2025 - 45th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 17-21, 2025, Proceedings, Part I. Lecture Notes in Computer Science*, vol. 16000, pp. 444–476. Springer (2025). https://doi.org/10.1007/978-3-032-01855-7_15, https://doi.org/10.1007/978-3-032-01855-7_15 2
10. Chen, Y., Nguyen, P.Q.: Bkz 2.0: Better lattice security estimates. In: Lee, D.H., Wang, X. (eds.) *Advances in Cryptology – ASIACRYPT 2011*. pp. 1–20. Springer Berlin Heidelberg, Berlin, Heidelberg (2011) 2
11. D’Anvers, J.P., Karmakar, A., Roy, S.S., Vercauteren, F.: Saber: Module-LWR based key exchange, CPA-secure encryption and CCA-secure KEM. In: Joux, A., Nitaj, A., Rachidi, T. (eds.) *AFRICACRYPT 18. LNCS*, vol. 10831, pp. 282–305. Springer, Cham (May 2018). https://doi.org/10.1007/978-3-319-89339-6_16 1
12. Ducas, L., Pulles, L.N.: Does the dual-sieve attack on learning with errors even work? In: Handschuh, H., Lysyanskaya, A. (eds.) *Advances in Cryptology - CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023, Proceedings, Part III. Lecture Notes in Computer Science*, vol. 14083, pp. 37–69. Springer (2023). https://doi.org/10.1007/978-3-031-38548-3_2, https://doi.org/10.1007/978-3-031-38548-3_2 2
13. Guo, Q., Johansson, T.: Faster dual lattice attacks for solving LWE with applications to CRYSTALS. In: Tibouchi, M., Wang, H. (eds.) *Advances in Cryptology - ASIACRYPT 2021 - 27th International Conference on the Theory and*

- Application of Cryptology and Information Security, Singapore, December 6-10, 2021, Proceedings, Part IV. Lecture Notes in Computer Science, vol. 13093, pp. 33–62. Springer (2021). https://doi.org/10.1007/978-3-030-92068-5_2, https://doi.org/10.1007/978-3-030-92068-5_2 2
14. ISO/IEC: Information technology — security techniques — encryption algorithms — part 2: Asymmetric ciphers — amendment 2. Standard ISO/IEC 18033-2:2006/Amd 2:2026, International Organization for Standardization, Geneva, Switzerland (Jun 2026), <https://www.iso.org/standard/86890.html> 1
 15. MATZOV: Report on the security of LWE: Improved dual lattice attack. Report, Zenodo (Apr 2022). <https://doi.org/10.5281/zenodo.6493704> 2
 16. Melo, V.D., Buchanan, W.J.: Kyfrog: A high-security lwe-based kem inspired by ml-kem (2025), <https://arxiv.org/abs/2512.06411> 2
 17. National Institute of Standards and Technology: Post-quantum cryptography: Round 3 submissions. <https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-3-submissions> (2020), accessed: 2026-07-27 1
 18. National Institute of Standards and Technology: Module-lattice-based digital signature standard. Tech. Rep. Federal Information Processing Standards Publication (FIPS) 204, U.S. Department of Commerce, Washington, D.C. (Aug 2024). <https://doi.org/10.6028/NIST.FIPS.204>, <https://doi.org/10.6028/NIST.FIPS.204> 1
 19. National Institute of Standards and Technology: Module-lattice-based key-encapsulation mechanism standard. Tech. Rep. Federal Information Processing Standards Publication (FIPS) 203, U.S. Department of Commerce, Washington, D.C. (Aug 2024). <https://doi.org/10.6028/NIST.FIPS.203>, <https://doi.org/10.6028/NIST.FIPS.203> 1
 20. National Institute of Standards and Technology: Stateless hash-based digital signature standard. Tech. Rep. Federal Information Processing Standards Publication (FIPS) 205, U.S. Department of Commerce, Washington, D.C. (Aug 2024). <https://doi.org/10.6028/NIST.FIPS.205>, <https://doi.org/10.6028/NIST.FIPS.205> 1
 21. The FrodoKEM Team: FrodoKEM: Practical quantum-secure key encapsulation from generic lattices. <https://frodokem.org/#news> (2026), news section. Accessed: 2026-07-27 1